API REST

Par Robin Delbaere

Objectifs

- **Comprendre et concevoir** une API REST
- **Protéger et contrôler** l’accès à une API
- Concevoir des API **robustes, fiables et maintenables**

Outillage

- Un **navigateur web**
- Un **client HTTP**
  - Postman
  - Insomnia
- Un **éditeur de code**
  - VS Code
  - Jetbrains IDE
- **NodeJS**

Introduction

Retour sur les fondamentaux

Qu’est-ce qu’une API ?

- **Interface de communication** entre deux composants
- Expose des **données** et des **actions**
- Définit un **contrat** d'utilisation
- Différents **standards** et **approches**
  - SOAP
  - REST
  - GraphQL

Le protocole HTTP

- Protocole **client / serveur** du Web
- Repose sur un échange **requête / réponse**
- Composantes d’une **requête**
  - URL
  - Méthode
  - En-têtes
  - Corps
- Composantes d’une **réponse**
  - Code de statut
  - En-têtes
  - Corps

Le format JSON

- Format de **données textuel**
- Basé sur des **objets** et des **tableaux**
- **Format d'échange** courant pour les API
- **Types** de donnée
  - Chaîne
  - Nombre
  - Booléen
  - Objet
  - Tableau
  - null

```json
{
    "id": 1042,
    "username": "john_doe",
    "email": "john@doe.com",
    "is_verified": true,
    "avatar_url": null,
    "skills": ["PHP", "Docker", "REST"],
    "preferences": {
        "theme": "dark",
        "notifications_enabled": false
    }
}
```

Activité - Exploration d'une API

Objectif

Créer une collection de requêtes sur Postman ou Insomnia

- Récupérer **une recette**
- Récupérer les **5 meilleures recettes**
- **Ajouter** une recette
- **Supprimer** une recette
- **Modifier** le nom, les ingrédients et la difficulté d'une recette

Liens utiles

- Postman
- Insomnia
- Documentation de l'API

Standard REST

Comprendre et concevoir

Les principes de REST

- Style **architectural**
- Interface **uniforme**
- Centré sur les **ressources**
- Communication **stateless**
- Repose sur **HTTP**

Les ressources

- Représente une **entité métier**
  - Clients
  - Produits
  - Commandes
- Identifiée par une **URI**
  - Collection : `/customers`, `/products`
  - Élément individuel : `/customers/51`, `/products/18`
- Parfois avec des **relations**
  - `/customers/51/orders`
  - `/orders/12/items`

Les méthodes

- Consulter une ressource
  - `GET /users`
  - `GET /users/42`
- Créer une ressource
  - `POST /users`
- Modifier une ressource
  - `PUT /users/42`
  - `PATCH /users/42`
- Supprimer une ressource
  - `DELETE /users/42`

Activité - Notre première API

Objectif

Créer une API REST avec **Node.js** et **Express** permettant de gérer des produits.

Les produits sont définis par : un **nom**, une **description**, un **prix** et une **catégorie**.

Implémenter les actions suivantes : **lister**, **consulter**, **ajouter**, **modifier**, **remplacer** et **supprimer**.

Contraintes

- Utiliser les méthodes **HTTP** appropriées
- Échanger les données au format **JSON**
- Retourner les **codes de statut HTTP** adaptés

Protéger et contrôler

Authentification, autorisation et limitation

Authentification vs autorisation

Authentification

Récupérer et confirmer **l'identité de l'utilisateur**

Autorisation

Déterminer ce qu'il est **autorisé à faire**

Méthodes d'authentification

Authentification basique

- Envoi d'un **identifiant et mot de passe**
- Transmis dans le header `Authorization`

Clé d'API

- **Clé secrète** associée à un usage
- Transmise dans un **header**
- Parfois en **paramètre d'URL**

Jeton d'accès

- Obtention d'un **jeton temporaire**
- Transmis avec **chaque requête**
- Au format `Authorization: Bearer <token>`

JSON Web Token

- **Format de jeton**
- Contient des informations **lisibles et vérifiables**
- Protégé par une **signature**

Activité - Authentification par jeton

Objectif

Ajouter une authentification par **jeton** à notre API.

- Créer un point d'entrée **d'authentification**
- Générer et retourner un **jeton d'accès**
- Limiter l'accès aux **actions d'écriture** aux utilisateurs authentifiés

Contraintes

- Un **jeton valide** est nécessaire pour chaque requête protégée
- Les jetons **expirent après 5 minutes**
- Retourner les **codes de statut HTTP** adaptés

Contrôle d'accès

- Contrôles d'accès **côté API**
  - Ne jamais faire confiance au client
  - Vérifier les droits à chaque requête
- Les niveaux de contrôle
  - Le **type de ressource**
  - La **ressource spécifique**
  - **L'action** effectuée
- Les mécanismes de contrôle
  - **Hiérarchie de rôles**
  - **Propriété** et **droits d'accès**
  - Principe du **moindre privilège**

Activité - À chacun ses droits

Contexte

HealthLab est une plateforme de gestion des examens biologiques pour les établissements de santé

- Les **professionnels de santé** transmettent les demandes d'analyses
- Les **équipes de laboratoire** réalisent les analyses et transmettent les résultats
- Les **patients** disposent d'un portail pour consulter leurs données
- Chaque analyse est **facturée au patient**

Objectifs

- Isoler les différentes **ressources**
- Définir les **rôles** et leur **hiérarchie**
- Déterminer les **droits d'accès** aux ressources et aux actions

Activité - Les rôles en action

Objectif

Ajouter un **système de rôles** pour contrôler l'accès aux actions.

Contraintes

- Les clients peuvent consulter les produits
- Les managers peuvent ajouter des produits
- Les managers peuvent modifier les produits qu'ils ont créés
- Les administrateurs peuvent ajouter, modifier et supprimer les produits

Validation des données

- Toujours valider les **données entrantes**
- Validation de **structure**
  - Type de données
  - Longueur
  - Format attendu
- Validation de **contenu**
  - Valeurs autorisées
  - Cohérence des données
  - Règles métier

Activité - Validation des données

Objectif

Ajouter une **validation des données** reçues par l'API

- Déterminer les **contraintes** à appliquer
- Installer **Joi**
- Implémenter la **validation des données**

Lien utile

- Joi

Protections complémentaires

- Toujours utiliser **HTTPS**
  - Sur les **environnements ouverts**
  - Pour **chiffrer** les échanges
- Configurer la politique **CORS**
  - Contrôle des **domaines autorisés**
  - Appliqué par les **navigateurs**
- Limiter le **trafic**
  - Limitation du **nombre de requêtes**
  - Dans le **temps** et par **usager**

Activité - Limiter les requêtes

Objectif

Ajouter une **limitation du nombre de requêtes** reçues par l'API

- Maximum **5 requêtes par minute et par adresse IP** pour la connexion
- Maximum **30 requêtes par minute et par adresse IP** pour tous les points d'entrée

Lien utile

- express-rate-limit

Bonnes pratiques

Qualité, fiabilité et évolutivité

Convention et cohérence

- Respecter les **standards HTTP**
  - Utiliser les **verbes cohérents**
  - Retourner un **code de statut approprié**
- Utiliser un **nommage homogène**
  - Pour les **ressources** et les **modèles de données**
  - Définir une **convention de casse**
  - Utiliser **une seule langue**
- Standardiser la **structure des réponses**
  - Utiliser une **enveloppe uniforme**
  - Normaliser **les formats de données**

Activité - Designer une API

Contexte

L'application **CityAddict** permet de **découvrir, rechercher et noter des lieux de sortie** : restaurants, bars, cafés.

L'utilisateur peut également créer des **listes de lieux**, publique ou privée ainsi que gérer ses **favoris**.

Les administrateurs peuvent **gérer les lieux** et **modérer les contenus**.

Objectif

Rédiger un **document de spécification**

- Identifier les **ressources** et leurs relations
- Définir les **points d'entrée**, leurs méthodes et leurs paramètres
- Concevoir les **structures de données entrantes**
- Concevoir les **structures de réponses**

Documentation

- Une **source de vérité** indispensable
  - Définir le **contrat de l'API**
  - Faciliter son **intégration** par des tiers
- Un contenu **exhaustif**
  - **Référentiel** des points d'entrée, méthodes, paramètres
  - **Exemples** de requête / réponse
  - **Règles métier** pertinentes
  - **Cas d'erreurs** possibles
- Un **cycle de vie** maîtrisé
  - **Générer** depuis le code
  - **Versionner** et faire **évoluer**
  - **Communiquer** sur les changements
- **L'accessibilité** en ligne de mire
  - Utiliser le standard `OpenAPI`
  - Proposer une interface `Swagger UI`

Activité - Documenter l'API

Objectif

Ajouter une **documentation interactive** à l'API

- Installer les bibliothèques
- Décrire les points d'entrée avec **OpenAPI**
- Générer automatiquement la **spécification**
- Intégrer **Swagger UI**

Liens utiles

- swagger-jsdoc
- swagger-ui-express

Performance

- Limiter les **données transférées**
  - **Restreindre** les champs sur les collections
  - Permettre de **personnaliser la réponse**
- **Paginer** les collections
  - Contrôler le **nombre de résultats**
  - Permettre de **parcourir les résultats suivants**
- Exploiter le **cache**
  - Définir des **stratégies d'expiration** via `Cache-Control`
  - Valider **l'état de la ressource** avec `ETag`
  - Mettre en place un **cache serveur**

Activité - Optimiser l'API

Objectifs

- **Paginer la collection** de produits
- **Exclure la description** dans les résultats de la collection
- Permettre de **sélectionner les champs** lors de la récupération d'un produit

Aller plus loin

- Observabilité & Surveillance
  - **Surveiller l'état** du service
  - **Historiser** les requêtes
  - Faciliter le **diagnostic**
- Gestion des versions
  - **Choisir** une version : `https://api.dummy.com/v1/users`
  - **Décommissionner** progressivement
- Évolutivité & Qualité
  - Maintenir une **suite de tests**
  - Déployer un flux **d'intégration continue**

Activité - Tracer les requêtes

Objectif

Mettre en place un système de **traçabilité des requêtes**

- Générer un **identifiant unique** pour chaque requête
- **Retourner cet identifiant** avec la réponse
- **Stocker l'identifiant et les informations** de la requête

Pour terminer

C'est bientôt fini

En résumé

Le cadre REST

- Style architectural **agnostique**
- Basé sur les **ressources** et les standards `HTTP`
- **S'adapte** à de nombreuses problématiques

> Une API RESTful applique l'ensemble des principes

Enjeux d'une API

- Disponibilité et réactivité
- Protection des données
- Simplicité d'intégration
- Évolutivité

Ressources

Concepts

- Guide HTTP - MDN
- REST - Roy Fielding

Standardisation

- Spécification OpenAPI
- Swagger

Outillage

- API Platform
- FastAPI
- NestJS